



Hands-On Lab Exercise 4: User Input in Shell Script

Topic: Taking User Input in Shell Script

Author: Dr. Sandeep Kumar Sharma

This lab explains how a shell script can **interact with the user** by taking input from the keyboard.

User input makes scripts **interactive, dynamic, and real-world usable**.

Without user input, scripts are static.

With user input, scripts become **tools and applications**.



What is User Input?

User input means:

Taking data from the user while the script is running.

Example: - Name - Age - File name - Path - Choice - Numbers



Why Do We Need User Input?

We need user input because: - Scripts must be dynamic - Users should control execution - Scripts must work in different situations - Automation should be flexible - Real systems require interaction



Purpose of User Input

User input helps scripts to: - Accept user decisions - Take values dynamically - Control script flow - Build interactive tools - Create real automation systems - Build CLI-based applications



Learning Objective

- Understand user input concept

- Learn how to take input
 - Use input in scripts
 - Build interactive scripts
 - Create user-driven automation
-

Learning Outcome

After this lab, participants will: - Take user input using `read` - Use input in commands - Build interactive scripts - Control script execution - Create real CLI tools

Command Used for User Input

```
read variable_name
```

Example:

```
read name
```

This stores user input into a variable.

Example 1: Basic User Input Script

Step 1: Create Script

```
touch input1.sh
```

Step 2: Open File

```
nano input1.sh
```

Step 3: Write Script

```
#!/bin/bash

echo "Enter your name:"
read name

echo "Hello $name, welcome to Shell Scripting"
```

Step 4: Permission

```
chmod +x input1.sh
```

Step 5: Run

```
./input1.sh
```



Example 2: Multiple User Inputs

Step 1: Create Script

```
touch input2.sh
```

Step 2: Open File

```
nano input2.sh
```

Step 3: Write Script

```
#!/bin/bash

echo "Enter your name:"
read name

echo "Enter your city:"
read city

echo "Enter your profession:"
read role

echo "User Details"
echo "Name: $name"
echo "City: $city"
echo "Profession: $role"
```

Step 4: Permission

```
chmod +x input2.sh
```

Step 5: Run

```
./input2.sh
```



Example 3: User Input for System Task

Step 1: Create Script

```
touch input3.sh
```

Step 2: Open File

```
nano input3.sh
```

Step 3: Write Script

```
#!/bin/bash

echo "Enter directory name to create:"
read dirname

mkdir $dirname

echo "Directory $dirname created successfully"
```

Step 4: Permission

```
chmod +x input3.sh
```

Step 5: Run

```
./input3.sh
```



Example 4: User Input + Automation

Step 1: Create Script

```
touch input4.sh
```

Step 2: Open File

```
nano input4.sh
```

Step 3: Write Script

```
#!/bin/bash

echo "Enter project name:"
read project

mkdir $project
mkdir $project/app
mkdir $project/logs
mkdir $project/config

echo "Project structure created for $project"
```

Step 4: Permission

```
chmod +x input4.sh
```

Step 5: Run

```
./input4.sh
```

Understanding the Flow

1. Script asks input
2. User types value
3. Value is stored in variable
4. Variable is used in commands
5. Script performs action

Key Understanding

- `read` takes user input
- Input is stored in variable
- Variables control execution
- Scripts become interactive
- Automation becomes flexible

Summary

- User input = interaction
- `read` command is used
- Scripts become dynamic
- User controls script behavior
- Foundation of real tools

End of Lab

This lab converts shell scripts into **interactive programs**.

Now scripts are not just automation — They are **user-driven systems**.

This is the foundation of: - CLI tools - DevOps tools - Automation engines - Interactive scripts

Author:

Dr. Sandeep Kumar Sharma



LinkedIn:

linkedin.com/in/sandeep-kaushik-2a345856

Medium Blog:

medium.com/@shyamsandeep28